

Tarea 2

Redes Recurrentes, Transformers y LLMs

IIC3697 – Aprendizaje Profundo

Jorge Ignacio Dimter Arce

2 de junio de 2026

§ 0 Índice

1. Parte 1 – Generación de Resúmenes de Noticias	2
1.1. Actividad 1 – Fundamentos Teóricos	2
1.1.1. Limitaciones de una RNN vanilla en textos largos	2
1.1.2. El mecanismo de atención en Seq2Seq	3
1.1.3. Encoder-only vs. encoder-decoder para generación de resúmenes	3
1.1.4. Métricas ROUGE y su preferencia sobre accuracy	3
1.2. Actividad 2 – LSTM Encoder-Decoder como Baseline	4
1.2.1. Descripción del modelo y del pipeline	4
1.2.2. Resultados y análisis de resúmenes generados	5
1.3. Actividad 3 – Seq2Seq con Mecanismo de Atención	6
1.3.1. Descripción de la arquitectura	6
1.3.2. Comparación cualitativa con el baseline LSTM	6
1.3.3. Análisis de los pesos de atención	7
1.4. Actividad 4 – Fine-tuning de BART y T5	7
1.4.1. Pipeline de fine-tuning con HuggingFace	7
1.4.2. Resultados comparativos	8
1.4.3. Análisis: ¿por qué BART supera a todos?	8
2. Parte 2 – Face Recognition con Contrastive Learning	9
2.1. Actividad 5 – Fundamentos Teóricos	9
2.1.1. ¿Por qué no usar softmax en reconocimiento facial?	9
2.1.2. Contrastive loss vs. triplet loss	9
2.1.3. Significado geométrico de la cercanía en el espacio de embeddings	10
2.2. Actividad 6 – Implementación de la Red Siamesa	10
2.2.1. Arquitectura y decisiones de diseño	10
2.3. Actividad 7 – Evaluación y Visualización	11
2.3.1. Curva ROC y determinación del umbral óptimo	11
2.3.2. Visualización t-SNE del espacio de embeddings	12
2.3.3. Análisis de errores: falsos positivos y falsos negativos	13

§ 1 Parte 1 – Generación de Resúmenes de Noticias

El objetivo de esta parte es explorar distintos enfoques para la tarea de *summarization* automática de noticias, utilizando el dataset CNN/DailyMail. Se parte desde un modelo LSTM entrenado desde cero como baseline, se incorpora un mecanismo de atención, y finalmente se realiza fine-tuning de modelos preentrenados de gran escala (BART y T5). Esta progresión permite comparar el impacto de cada mejora en la arquitectura y del preentrenamiento sobre la calidad de los resúmenes generados.

El dataset se carga mediante la librería `datasets` de HuggingFace, seleccionando 10.000 ejemplos de entrenamiento, 1.000 de validación y 500 de test. Cada ejemplo contiene un campo `article` (la noticia completa) y un campo `highlights` (el resumen de referencia con viñetas). Se construye un vocabulario propio de hasta 30.000 palabras, tokenizando con una expresión regular que captura palabras alfanuméricas y signos de puntuación. Los artículos se truncan a 400 tokens y los resúmenes a 100, y se añaden tokens especiales `<sos>`, `<eos>`, `<pad>` y `<unk>`.

1.1. Actividad 1 – Fundamentos Teóricos

1.1.1. Limitaciones de una RNN vanilla en textos largos

Una red neuronal recurrente vanilla (RNN) procesa secuencias de forma iterativa: en cada paso t actualiza su estado oculto $h_t = f(h_{t-1}, x_t)$. En teoría, este estado debería acumular toda la información relevante de la secuencia hasta ese punto. En la práctica, sin embargo, surgen dos problemas fundamentales al trabajar con textos largos como noticias.

El primero son los vanishing gradients (o exploding). Durante el entrenamiento, el gradiente de la pérdida se propaga hacia atrás a través del tiempo (back propagation). En secuencias largas, el gradiente se multiplica repetidamente por los pesos recurrentes: si estos pesos tienen valores menores a 1 en módulo, el gradiente colapsa a cero y las capas tempranas dejan de aprender (gradiente evanescente); si tienen valores mayores a 1, el gradiente crece exponencialmente y desestabiliza el entrenamiento (gradiente explosivo). Las LSTMs mitigan este problema mediante compuertas, pero no lo eliminan completamente en secuencias muy largas.

La segunda limitación es el cuello de botella del vector de contexto fijo. En una arquitectura encoder-decoder, el encoder comprime toda la secuencia de entrada en un único vector de dimensión fija, que luego sirve como estado inicial del decoder. Para artículos de noticias que pueden superar las 800 palabras, es imposible codificar toda la información relevante en un vector de, por ejemplo, 512 dimensiones. Como señalan Bahdanau et al., el rendimiento del encoder-decoder básico se deteriora rápidamente conforme aumenta la longitud de la entrada. Esta es la motivación central del mecanismo de atención desarrollado en la Actividad 3.

1.1.2. El mecanismo de atención en Seq2Seq

El mecanismo de atención resuelve el cuello de botella al eliminar la restricción de comprimir toda la entrada en un único vector. En lugar de eso, el encoder produce una secuencia de estados ocultos $\bar{h}_1, \bar{h}_2, \dots, \bar{h}_T$, uno por cada token del artículo. En cada paso de decodificación t , el mecanismo de atención calcula un vector de contexto c_t como una suma ponderada de todos estos estados del encoder:

$$e_{t,s} = v^\top \tanh(W[h_t; \bar{h}_s]), \quad \alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}, \quad c_t = \sum_s \alpha_{t,s} \bar{h}_s$$

Los pesos $\alpha_{t,s}$ indican cuánta “atención” presta el decoder, en el paso t , a la posición s del artículo. Esto permite que el modelo acceda selectivamente a distintas partes del artículo en cada paso de generación, atacando directamente la limitación del vector fijo.

1.1.3. Encoder-only vs. encoder-decoder para generación de resúmenes

Los modelos **encoder-only** como BERT procesan la secuencia bidireccionalmente y producen representaciones contextuales de cada token. Sin embargo, no son autorregresivos: no pueden generar texto nuevo de forma natural, ya que sus predicciones son independientes entre sí. Son ideales para tareas de comprensión (clasificación, *named entity recognition*, respuesta a preguntas extractiva).

Los modelos **encoder-decoder** como BART y T5 combinan un encoder bidireccional con un decoder autoregresivo. El encoder construye representaciones ricas del texto de entrada, y el decoder genera el texto de salida token por token condicionado en dichas representaciones mediante *cross-attention*. Esta arquitectura es la más apropiada para resúmenes abstractivos porque: (i) requiere *comprender* el documento fuente (encoder bidireccional), (ii) requiere *generar* texto fluido de longitud variable (decoder autoregresivo), y (iii) puede producir frases que no aparecen literalmente en el artículo.

1.1.4. Métricas ROUGE y su preferencia sobre accuracy

Las métricas ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*) son el estándar para evaluar resúmenes automáticos. Miden la superposición léxica entre el texto generado y uno o más resúmenes de referencia:

- **ROUGE-1**: proporción de unigramas en común entre generado y referencia.
- **ROUGE-2**: proporción de bigramas en común; captura algo de fluidez local y coherencia de frases.
- **ROUGE-L**: basada en la subsecuencia común más larga (*LCS*); considera el orden relativo sin requerir contigüidad exacta.

La *accuracy* no es aplicable aquí porque requeriría que el modelo genere exactamente las mismas palabras en el mismo orden que la referencia, lo cual es prácticamente imposible en generación libre. Para un mismo artículo, existen múltiples resúmenes igualmente

válidos con distintas palabras. ROUGE cuantifica el solapamiento léxico de forma más flexible, aunque tiene sus propias limitaciones: no captura coherencia semántica ni fluidez gramatical. En la práctica se reportan las tres métricas en conjunto como F1, tal como hacen Raffel et al. en CNN/DailyMail.

1.2. Actividad 2 – LSTM Encoder-Decoder como Baseline

1.2.1. Descripción del modelo y del pipeline

La arquitectura baseline consiste en un `LSTMSummarizer` con dos componentes principales: un encoder LSTM que lee el artículo y lo comprime en un estado oculto (h, c) , y un decoder LSTM que parte de ese estado y genera el resumen token por token. La arquitectura es la siguiente:

```
class LSTMSummarizer(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        self.embedding = nn.Embedding(vocab_size, embed_dim,
                                       padding_idx=PAD_IDX)
        self.encoder = nn.LSTM(embed_dim, hidden_dim, batch_first=
                                True)
        self.decoder = nn.LSTM(embed_dim, hidden_dim, batch_first=
                                True)
        self.fc_out = nn.Linear(hidden_dim, output_dim)
```

Listing 1: Arquitectura del LSTM baseline

Durante el entrenamiento se usa *teacher forcing* con probabilidad 0.5: en cada paso, con probabilidad 0.5 el decoder recibe el token correcto del resumen de referencia como entrada (en lugar de su propia predicción anterior). Esto acelera la convergencia, pero genera una brecha entre el entrenamiento y la inferencia. La pérdida es la entropía cruzada ignorando el token de padding (`ignore_index=PAD_IDX`), y el gradiente se recorta a norma máxima 1.0 para estabilizar el entrenamiento.

Los hiperparámetros principales se resumen en la Tabla 1.

Tabla 1: Hiperparámetros del modelo LSTM baseline

Parámetro	Valor	Rol
EMBED_DIM	256	Dimensión de los embeddings de palabras
HIDDEN_DIM	512	Dimensión del estado oculto del LSTM
BATCH_SIZE	32	Tamaño de minilote
EPOCHS	5	Épocas de entrenamiento
LR	10^{-3}	Tasa de aprendizaje (Adam)
TEACHER_FORCING	0.5	Prob. de usar token de referencia en decoder
MAX_ARTICLE_LEN	400	Tokens máximos del artículo
MAX_SUMMARY_LEN	100	Tokens máximos del resumen
MAX_VOCAB	30.000	Tamaño máximo del vocabulario

1.2.2. Resultados y análisis de resúmenes generados

Al revisar los resúmenes generados por el LSTM en el conjunto de validación, se observa un patrón recurrente de baja calidad: secuencias repetidas de tokens frecuentes, y aparición frecuente de `<unk>` en lugar de nombres propios, cifras y entidades. Los resúmenes no mantienen fidelidad al contenido del artículo.

Estas limitaciones tienen causas directas:

1. **Cuello de botella del estado oculto.** Toda la noticia (hasta 400 tokens) se comprime en un único par (h, c) de dimensión 512. Para artículos largos, esta representación no puede retener todos los detalles relevantes, y el decoder parte de un punto de información incompleta.
2. **Vocabulario cerrado.** Nombres propios, topónimos y cifras que no aparecen entre las 30.000 palabras más frecuentes del training set se mapean a `<unk>`, y el modelo no tiene mecanismo para recuperarlos.
3. **Brecha de exposición (*exposure bias*).** Con teacher forcing 0.5, el modelo entrena con tokens de referencia correctos, pero en inferencia debe guiarse por sus propias (a veces erróneas) predicciones. Un error temprano en la secuencia generada se arrastra y amplifica.
4. **Escala de datos y cómputo.** 10.000 ejemplos y 5 épocas son insuficientes para que el modelo aprenda la distribución compleja del lenguaje periodístico. Los modelos de generación de texto de calidad se entrenan con órdenes de magnitud más datos.

Conclusión. El LSTM baseline aprende la *forma* superficial de un resumen (longitud aproximada, uso de puntuación), pero no su contenido. La causa estructural es el cuello de botella del vector fijo, que es precisamente lo que el mecanismo de atención viene a resolver en la siguiente actividad.

1.3. Actividad 3 – Seq2Seq con Mecanismo de Atención

1.3.1. Descripción de la arquitectura

Se implementa una arquitectura Seq2Seq compuesta por tres clases: **Encoder**, **Attention** y **Decoder**. La diferencia clave respecto al baseline es que el encoder ahora retorna todos sus estados ocultos intermedios (uno por cada token del artículo), no solo el estado final:

```
class Encoder(nn.Module):
    def forward(self, x):
        emb = self.embedding(x)
        outputs, (h, c) = self.lstm(emb)
        # outputs: (batch, len_articulo, hidden_dim)
        # cada fila es el estado oculto del token correspondiente
        return outputs, (h, c)
```

Listing 2: Encoder con salida de todos los estados ocultos

El módulo de atención implementa la variante aditiva de Bahdanau. En cada paso del decoder, concatena el estado oculto actual del decoder con cada estado del encoder, los proyecta a través de una capa lineal y calcula un score por posición del artículo. Aplicando softmax se obtienen los pesos de atención α , y el vector de contexto es la suma ponderada de los estados del encoder:

```
class Attention(nn.Module):
    def forward(self, hidden, encoder_outputs):
        hidden_rep = hidden.unsqueeze(1).repeat(1, src_len, 1)
        energy = torch.tanh(self.attn(torch.cat((hidden_rep,
            encoder_outputs), dim=2)))
        scores = self.v(energy).squeeze(2)           # (batch,
            len_articulo)
        weights = torch.softmax(scores, dim=1)       # pesos de
            atencion
        context = torch.bmm(weights.unsqueeze(1), encoder_outputs).
            squeeze(1)
        return weights, context
```

Listing 3: Módulo de atención aditiva

El decoder concatena el embedding del token actual con el vector de contexto antes de pasarlo por el LSTM, y la capa de salida combina el estado oculto del decoder con el vector de contexto para la predicción final. Esto enriquece la representación con información directa del artículo en cada paso.

1.3.2. Comparación cualitativa con el baseline LSTM

En términos de pérdida de entrenamiento, el Seq2Seq con atención muestra una reducción progresiva de $\sim 7,2$ a $\sim 5,3$ a lo largo de 5 épocas, lo que confirma que el modelo está aprendiendo. Sin embargo, la pérdida de validación se mantiene relativamente estable cerca de 7.0 e incluso tiende a subir, lo que sugiere que el modelo comienza a sobreajustar.

Cualitativamente, la calidad de los resúmenes generados sigue siendo pobre: se observan secuencias repetitivas de `<unk>` y signos de puntuación. El mecanismo de atención no alcanza a desarrollarse correctamente en solo 5 épocas y con 10.000 ejemplos. La diferencia teórica existe: el decoder *podría* mirar distintas partes del artículo en cada paso, pero el modelo no tuvo suficiente entrenamiento para aprender a hacerlo de forma útil.

1.3.3. Análisis de los pesos de atención

Se visualizaron los heatmaps de atención para tres noticias del conjunto de validación, donde cada fila del heatmap corresponde a una palabra generada y cada columna a una posición del artículo, con la intensidad del color indicando el peso $\alpha_{t,s}$.

Un modelo con atención bien entrenado mostraría una estructura variada por fila: distintas palabras generadas mirarían distintas partes del artículo. Lo que se observa en la práctica es lo contrario: la atención está casi siempre concentrada en las mismas 2–3 posiciones del artículo, independientemente de qué palabra se está generando. Esto se manifiesta como *columnas verticales brillantes* en el heatmap, no como una distribución que cambie por fila.

En `val_0`, hay un pico muy marcado alrededor de la posición 158 que domina la atención para prácticamente todas las palabras generadas; en `val_2`, el peak principal está cerca de la posición 75. Este patrón de atención degenerado es consistente con la baja calidad de los resúmenes: el modelo está “atascado” mirando siempre el mismo punto, incapaz de distribuir la atención de forma informativa. Con más épocas de entrenamiento y mejores técnicas de regularización, se esperaría que los pesos de atención comenzaran a mostrar la variación por fila característica de un modelo bien entrenado.

1.4. Actividad 4 – Fine-tuning de BART y T5

1.4.1. Pipeline de fine-tuning con HuggingFace

Para los modelos preentrenados se utiliza el framework `transformers` de HuggingFace. Se implementa una función `make_hf_dataset` que tokeniza los artículos y resúmenes con el tokenizador nativo de cada modelo, truncando a 512 tokens (fuente) y 128 tokens (objetivo). Para T5 se antepone el prefijo `"summarize: "` al artículo, ya que T5 usa este formato texto-a-texto para especificar la tarea.

El entrenamiento usa `Seq2SeqTrainer` con las siguientes configuraciones comunes: 3 épocas, batch efectivo de 32 (8 por dispositivo $\times$ 4 pasos de acumulación de gradiente), FP16 habilitado si hay GPU disponible, y `EarlyStoppingCallback` con paciencia 2 para evitar sobreajuste. Las tasas de aprendizaje difieren entre modelos: 3×10^{-5} para BART (modelo más grande, 139 M parámetros) y 5×10^{-5} para T5 (60 M parámetros). La evaluación ROUGE se computa sobre 500 ejemplos de test usando `rouge_scorer` con *stemming*.

1.4.2. Resultados comparativos

La Tabla 2 muestra los resultados de los cuatro modelos en las tres métricas ROUGE.

Tabla 2: Comparativa ROUGE – CNN/DailyMail ($N_{\text{train}} = 10,000$, $N_{\text{test}} = 500$)

Modelo	Paráms	Pre-entr.	Épocas	R-1	R-2	R-L
LSTM Enc-Dec	~10M	–	5	4.85	0.35	4.48
Seq2Seq + Atención	~12M	–	5	11.24	1.21	9.62
T5 (t5-small)	60M	Span-corr. (750 GB)	3	32.27	12.72	23.36
BART (bart-base)	139M	Denoising (160 GB)	3	32.89	12.16	22.89

1.4.3. Análisis: ¿por qué BART supera a todos?

La brecha entre los modelos entrenados desde cero (LSTM: 4.85, Seq2Seq: 11.24) y los preentrenados (BART: 32.89) es enorme, y no se explica por diferencias en hiperparámetros sino por el preentrenamiento. BART y T5 llegaron al fine-tuning con representaciones ricas del lenguaje aprendidas sobre cientos de gigabytes de texto; el fine-tuning solo tuvo que adaptar esas representaciones a la tarea de resumen. El LSTM y el Seq2Seq, en cambio, deben aprender el lenguaje *y* la tarea simultáneamente, con solo 10.000 ejemplos y 5 épocas.

La diferencia entre BART y T5 es más matizada. Ambos usan la misma arquitectura Transformer encoder-decoder, pero difieren en su estrategia de preentrenamiento:

- **BART** usa *denoising autoencoding*: el modelo recibe el texto con corrupciones (tokens borrados, spans enmascarados, oraciones permutadas) y debe reconstruir el texto original. Este objetivo está directamente alineado con la tarea de resúmenes, donde se “reconstruye” la esencia de un texto largo.
- **T5** usa un formato texto-a-texto unificado con *span corruption*: enmascara spans del texto y los predice. Es más genérico, lo que le da versatilidad, pero el alineamiento con la generación fluida es algo menor.

En términos prácticos, BART gana en las tres métricas ROUGE. Cualitativamente también genera resúmenes más fluidos y factualmente más precisos. T5 compensa con un dataset de preentrenamiento mucho mayor (750 GB vs. 160 GB), pero en generación de texto pura BART mantiene una ventaja derivada de su objetivo de entrenamiento.

Conclusión de la Parte 1. La elección de usar un modelo preentrenado importa mucho más que el ajuste de hiperparámetros. La progresión LSTM → Seq2Seq+Atención muestra que la atención mejora la arquitectura teóricamente, pero en condiciones de cómputo y datos limitados el beneficio observado es modesto. El salto real ocurre con el preentrenamiento masivo: BART logra ROUGE-1 de 32.89 con solo 3 épocas de fine-tuning, algo que los modelos desde cero no alcanzan ni con más entrenamiento.

§ 2 Parte 2 – Face Recognition con Contrastive Learning

El objetivo de esta parte es implementar un sistema de reconocimiento facial basado en contrastive learning. En lugar de entrenar un clasificador con softmax sobre identidades fijas, se busca aprender un espacio de embeddings donde la distancia euclidiana entre dos imágenes refleje su similitud de identidad. Se usa el dataset LFW, con 2.200 pares de entrenamiento y 1.000 de test, cada uno etiquetado como misma persona ($y = 1$) o personas distintas ($y = 0$).

2.1. Actividad 5 – Fundamentos Teóricos

2.1.1. ¿Por qué no usar softmax en reconocimiento facial?

Un clasificador softmax aprende a distinguir entre un conjunto *fijo y cerrado* de identidades. Dados N individuos conocidos, el modelo aprende N vectores de clase y asigna cada imagen a la clase más probable. Este enfoque tiene una limitación fundamental: si aparece un individuo nuevo (no visto durante el entrenamiento), el modelo no puede reconocerlo, ya que no existe una clase para él. Añadir una identidad nueva requeriría reentrenar todo el clasificador.

El aprendizaje métrico resuelve esto al separar la *representación* de la *comparación*: el modelo aprende a proyectar imágenes a un espacio de embeddings donde imágenes de la misma persona quedan cercanas, sin importar si esa persona fue vista durante el entrenamiento. Reconocer a alguien se reduce entonces a comparar distancias en ese espacio, lo que generaliza naturalmente a identidades nuevas.

2.1.2. Contrastive loss vs. triplet loss

Ambas funciones de pérdida buscan lo mismo: acercar embeddings de la misma persona y alejar los de personas distintas. Difieren en la granularidad de la comparación:

Contrastive loss opera sobre *pares* (x_1, x_2, y) :

$$\mathcal{L}(y, d) = y \cdot d^2 + (1 - y) \cdot \max(0, m - d)^2$$

donde $d = \|f(x_1) - f(x_2)\|_2$ y m es un margen. Minimiza d para pares positivos y maximiza d (hasta el margen) para pares negativos. Es simple de implementar y apropiada cuando los datos ya vienen organizados en pares, como en LFW.

Triplet loss opera sobre *tríos* (x_a, x_p, x_n) : ancla, positivo (misma persona) y negativo (persona distinta):

$$\mathcal{L} = \max(0, \|f(x_a) - f(x_p)\|^2 - \|f(x_a) - f(x_n)\|^2 + m)$$

Proporciona más contexto relativo al modelo y genera gradientes más informativos. Es preferible cuando se dispone de muchas identidades con pocos ejemplos por persona, aunque requiere una estrategia cuidadosa de *hard negative mining* para que el entrenamiento no colapse.

2.1.3. Significado geométrico de la cercanía en el espacio de embeddings

Cada imagen se transforma en un punto en un espacio de alta dimensión ($\mathbb{R}^{128}$ en esta implementación). La distancia euclidiana entre dos puntos refleja la disimilitud de identidad: pequeña si son la misma persona, grande si son distintas. Gracias a la normalización L2 de los embeddings, las distancias están acotadas y son comparables entre pares.

Durante el entrenamiento, la contrastive loss empuja los embeddings de la misma persona hacia regiones compactas del espacio y los de distintas personas hacia regiones separadas por al menos el margen m . En la práctica, el umbral de decisión ($d^* = 0,409$ en este experimento) se determina optimizando el índice de Youden sobre la curva ROC del conjunto de test (se probaron también otras formas de obtener el umbral de decisión obteniendo resultados muy similares).

2.2. Actividad 6 – Implementación de la Red Siamesa

2.2.1. Arquitectura y decisiones de diseño

La red siamesa usa ResNet18 preentrenado en ImageNet como backbone. Todos los parámetros del backbone se congelan: solo se entrena la cabeza de proyección que reemplaza la capa fully-connected original. Esta cabeza transforma los 512 features de ResNet18 a 256 dimensiones con ReLU y Dropout(0.5), y finalmente a 128 dimensiones. Los embeddings se normalizan con norma L2.

```
backbone.fc = nn.Sequential(
    nn.Linear(in_features, 256), # in_features=512 en ResNet18
    nn.ReLU(inplace=True),
    nn.Dropout(0.5), # aumentado de 0.3 a 0.5 para mejor
                      generalizacion
    nn.Linear(256, embedding_dim), # embedding_dim=128
)
# Normalizacion L2 en forward_one:
emb = nn.functional.normalize(emb, p=2, dim=1)
```

Listing 4: Cabeza de proyección de la red siamesa

En total, el modelo tiene $\sim 11,5$ M de parámetros, de los cuales solo ~ 132 K (la cabeza de proyección) son entrenables.

La Tabla 3 detalla los hiperparámetros y su justificación.

Tabla 3: Hiperparámetros de la red siamesa

Hiperparámetro	Valor	Justificación
embedding_dim	128	Suficiente para codificar identidad; dimensiones mayores no ayudan con 2.200 pares
margin (m)	1.0	Separación mínima entre pares negativos; valor estándar en literatura
LR	3×10^{-4}	Backbone congelado; lr moderado para la cabeza de proyección
batch_size	32	Balance entre estabilidad del gradiente y memoria GPU
epochs	30	Dataset pequeño; convergencia observada antes de 20 épocas
weight_decay	10^{-4}	Regularización L2 para limitar sobreajuste
scheduler	StepLR($\gamma=0.5$, step=8)	Reduce lr cada 8 épocas para refinar

La contrastive loss se implementa directamente sobre la distancia euclidiana entre los embeddings de los dos miembros del par. Para pares positivos ($y = 1$) penaliza la distancia directamente; para pares negativos ($y = 0$) solo penaliza si la distancia es menor al margen m , forzando una separación mínima:

```
def forward(self, emb1, emb2, label):
    d = F.pairwise_distance(emb1, emb2, p=2) # distancia euclidiana
    loss_pos = label.float() * d.pow(2)
    loss_neg = (1 - label.float()) * torch.clamp(m - d, min=0).pow(2)
    return (loss_pos + loss_neg).mean()
```

Listing 5: Implementación de la contrastive loss

2.3. Actividad 7 – Evaluación y Visualización

2.3.1. Curva ROC y determinación del umbral óptimo

La evaluación consiste en calcular la distancia euclidiana entre los embeddings de cada par del conjunto de test, y usar esas distancias (negadas, para que mayor similitud corresponda a mayor score) para construir la curva ROC. El umbral óptimo se determina mediante el índice de Youden: el punto de la curva que maximiza $\text{TPR} - \text{FPR}$.

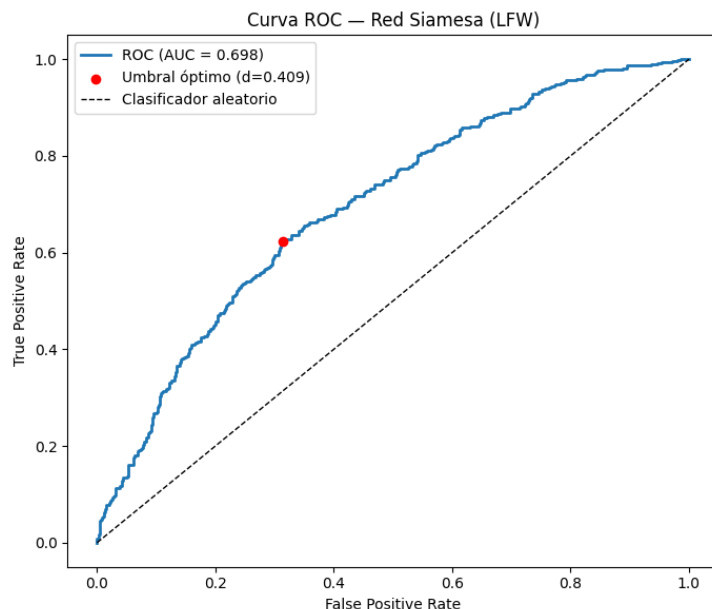


Figura 1: Curva ROC con umbral optimo

Tabla 4: Métricas de evaluación – Red Siamesa (LFW test set, 1.000 pares)

Métrica	Valor
AUC	0.6981
Umbral óptimo d^*	0.409
Accuracy con umbral óptimo	65.40 %
Total de Errores	346
Falsos positivos	157
Falsos negativos	189

2.3.2. Visualización t -SNE del espacio de embeddings

Para entender cualitativamente qué aprendió la red, se extraen los embeddings de 500 imágenes del test set y se proyectan a 2 dimensiones con t -SNE. Los puntos se colorean según la etiqueta del par de origen (1 = misma persona, 0 = personas distintas).

Lo primero que llama la atención es que *no hay una separación clara* entre los puntos rojos (misma persona) y azules (distintas). Ambos colores aparecen mezclados a lo largo de todo el espacio, sin formar clusters bien definidos por identidad. Esto es consistente con el número de errores observados.

Dicho esto, el gráfico tampoco es completamente aleatorio: hay zonas con cierta concentración de un color, lo que indica que el modelo aprendió algo pero no lo suficiente para una separación robusta. Las causas posibles son el tamaño reducido del dataset (2.200 pares), el hecho de que las etiquetas del t -SNE corresponden a pares y no a identidades

individuales, y las limitaciones del backbone congelado. Un modelo con fine-tuning del backbone completo y más datos debería mostrar clusters más definidos.

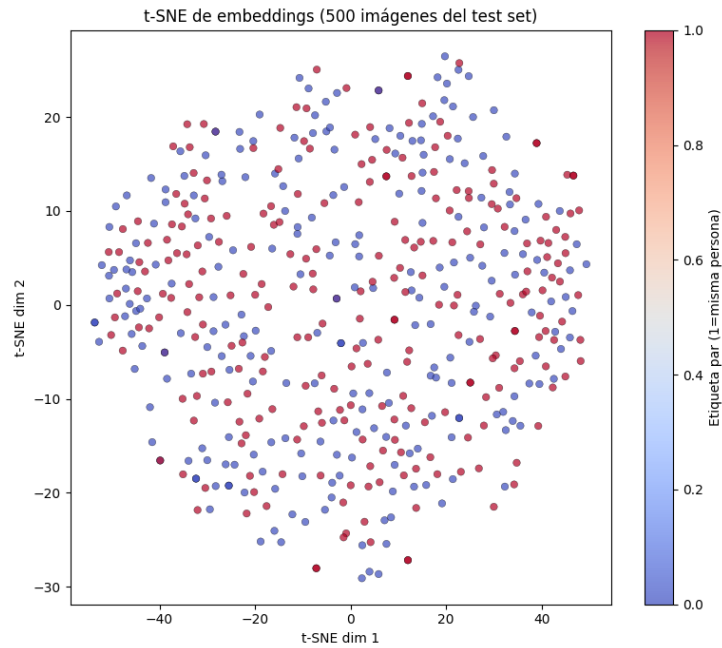


Figura 2: Proyección t-SNE para embeddings.

2.3.3. Análisis de errores: falsos positivos y falsos negativos

El modelo cometió 346 errores en total sobre los 1.000 pares de test: 157 falsos positivos y 189 falsos negativos. El análisis de los casos concretos revela patrones sistemáticos:

Falsos positivos (predijo misma persona, eran distintas): Las distancias están muy cerca del umbral (0.354, 0.363, 0.383 vs. $d^* = 0,409$). Los rostros comparten rasgos visuales generales: complexión similar, misma franja etaria, tono de piel parecido, y en algunos casos ángulos fotográficos similares. Esto sugiere que el modelo está capturando características de apariencia general (demografía, iluminación) en lugar de rasgos de identidad específicos y finos.

Falsos negativos (predijo distintas, eran la misma): Los errores son más severos: las distancias son bastante mayores al umbral (0.532 a 0.618), lo que indica que el modelo está bastante “convencido” de que son personas distintas. La causa visible es la variabilidad intra-personal: cambios importantes de iluminación, expresión facial y ángulo de la cámara entre las dos fotos de la misma persona alteran significativamente el embedding generado.

Patrón general de errores. Ambos tipos de errores apuntan a: el modelo es sensible a variaciones de iluminación y pose, y no siempre logra aprender rasgos de identidad suficientemente robustos e invariantes. Mejoras posibles: (i) descongelar parte del backbone para fine-tuning, (ii) aumentación de datos con cambios de iluminación y pose, (iii) usar triplet loss con hard negative mining, (iv) usar un backbone más potente (ResNet50 o ViT).

§ 2 Conclusiones Generales

En la **Parte 1**, la progresión LSTM $\rightarrow$ Seq2Seq+Atención $\rightarrow$ BART/T5 ilustra cómo el preentrenamiento masivo domina la calidad final. La atención mejora la arquitectura teóricamente, pero en condiciones de cómputo y datos limitados el beneficio es bajo. BART obtiene ROUGE-1 de 32.89, casi $3\times$ más que el Seq2Seq con atención (11.24) y casi $7\times$ más que el LSTM (4.85), con solo 3 épocas de fine-tuning.

En la **Parte 2**, la red siamesa con contrastive loss demuestra que el aprendizaje métrico es un enfoque viable y escalable para reconocimiento facial, capaz de generalizar a identidades no vistas durante el entrenamiento. Sin embargo, el tamaño reducido del dataset LFW (2.200 pares de entrenamiento) limita la capacidad del modelo para aprender representaciones discriminativas. El análisis de errores revela que el modelo es sensible a variaciones de iluminación y pose, lo que apunta a la necesidad de mejor aumentación de datos y posiblemente un backbone más potente con fine-tuning parcial.